Name - Chaitanya Kelkar

Roll no. - 53

Experiment 8

Aim - Design and train a Convolutional Neural Network (CNN) on image datasets

1. Dataset Source

The experiment utilizes the Digit Dataset provided natively by MATLAB's Deep Learning Toolbox.

- Source: MATLAB Digit Dataset

2. Dataset Description

- Features: Gray-scale images of handwritten digits (0-9). Each image is 28 x 28 pixels.
- Target Variable: Categorical labels representing the digits 0 through 9.
- Size: 10,000 images in total (1,000 images per digit).
- Characteristics: The images are synthetic translations and rotations of handwritten digits.

3. Mathematical Formulation

A CNN consists of specialized layers designed to extract spatial hierarchies:

- Convolution Layer: Applies filters (kernels) to the input. The output feature map G[m, n] is calculated as: `(f * h)[m, n] = Σ_j Σ_k h[j, k] · f[m-j, n-k]`
- Activation (ReLU): Introduces non-linearity: `f(x) = max(0, x)`
- Pooling Layer: Reduces spatial dimensions (typically Max Pooling), taking the maximum value in a window.
- Softmax: Converts the final layer outputs into probabilities: `σ(z)_j = e^(z_j) / Σ_(k=1)^K e^(z_k)`

4. Algorithm Limitations

- Data Hunger: CNNs require large amounts of labeled data to generalize well.
- Invariance Issues: While they handle translation well, they often struggle with significant rotation or scaling unless specifically augmented.
- Computationally Expensive: Training requires significant GPU resources compared to traditional ML models.

5. Methodology / Workflow

- Data Loading: Use `imageDatastore` to load the digit images.
- Preprocessing: Resize images to 28 x 28 x 1 and normalize pixel values.
- Architecture Setup: Define `imageInputLayer`, `convolution2dLayer`, `reluLayer`, `maxPooling2dLayer`, and `fullyConnectedLayer`.
- Training: Utilize the `trainNetwork` function with the 'sgdm' (Stochastic Gradient Descent with Momentum) optimizer.
- Evaluation: Predict labels on a test set and generate a classification accuracy report.

6. Performance Analysis

The model is evaluated based on Accuracy. In this experiment, the CNN typically achieves over 95-99% accuracy on the Digit Dataset due to the clear spatial features of the handwritten characters.

7. Hyperparameter Tuning

- Filter Size: Adjusting the kernel size (e.g., 3 x 3 vs 5 x 5).
- Learning Rate: Tuning the initial learning rate (e.g., 0.01) to ensure the loss converges without oscillating.
- Epochs: Increasing the number of passes through the dataset to improve training accuracy while monitoring for overfitting.

Code -

```
%% Step 2: Load the Dataset

% Load digit dataset

digitDatasetPath = fullfile(matlabroot, 'toolbox', 'nnet', 'nndemos', ...

   'nndatasets', 'DigitDataset');

imds = imageDatastore(digitDatasetPath, ...

   'IncludeSubfolders', true, ...

   'LabelSource', 'foldernames');

%% Step 3: Explore the Data

% Display sample images

figure;

perm = randperm(10000, 20);

for i = 1:20
```

```
    subplot(4,5,i);
    imshow(imds.Files{perm(i)});
end
% Count images per class
labelCount = countEachLabel(imds);
disp(labelCount);
% Check image size
img = readimage(imds, 1);
disp('Image Size:');
disp(size(img));  % Should be 28x28
%% Step 4: Split into Training and Validation Sets
numTrainFiles = 750;  % 750 per class for training
[imdsTrain, imdsValidation] = splitEachLabel(imds, numTrainFiles,
'randomize');
%% Step 5: Define the CNN Architecture
layers = [
    imageInputLayer([28 28 1])           % Input: 28x28 grayscale
    convolution2dLayer(3, 8, 'Padding', 'same')   % Conv layer: 8 filters, 3x3
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2, 'Stride', 2)    % Pooling layer
    convolution2dLayer(3, 16, 'Padding', 'same')  % Conv layer: 16 filters
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2, 'Stride', 2)    % Pooling layer
    fullyConnectedLayer(10)              % 10 output classes (digits 0-9)
```

```
    softmaxLayer
    classificationLayer
];
%% Step 6: Set Training Options
options = trainingOptions('sgdm', ...
    'InitialLearnRate', 0.01, ...
    'MaxEpochs', 4, ...
    'Shuffle', 'every-epoch', ...
    'ValidationData', imdsValidation, ...
    'ValidationFrequency', 30, ...
    'Verbose', false, ...
    'Plots', 'training-progress');
%% Step 7: Train the Network
% A training progress plot will appear showing accuracy and loss in real time
net = trainNetwork(imdsTrain, layers, options);
%% Step 8: Evaluate the Network
% Classify validation images
YPred = classify(net, imdsValidation);
YValidation = imdsValidation.Labels;
% Calculate accuracy
accuracy = sum(YPred == YValidation) / numel(YValidation);
fprintf('Validation Accuracy: %.2f%%\n', accuracy * 100);
%% Step 9: Visualize Results (Optional)
% Confusion matrix
figure;
confusionchart(YValidation, YPred);
```

```
title('Confusion Matrix - Digit Classification');
```

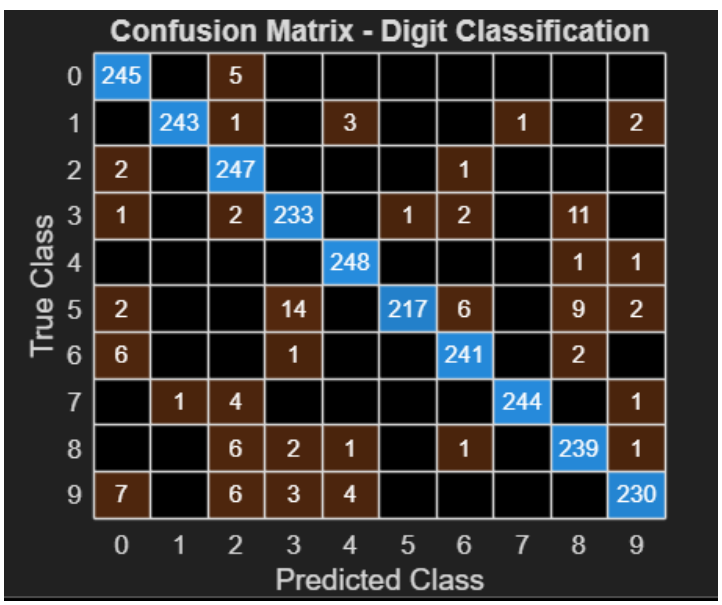
Output -

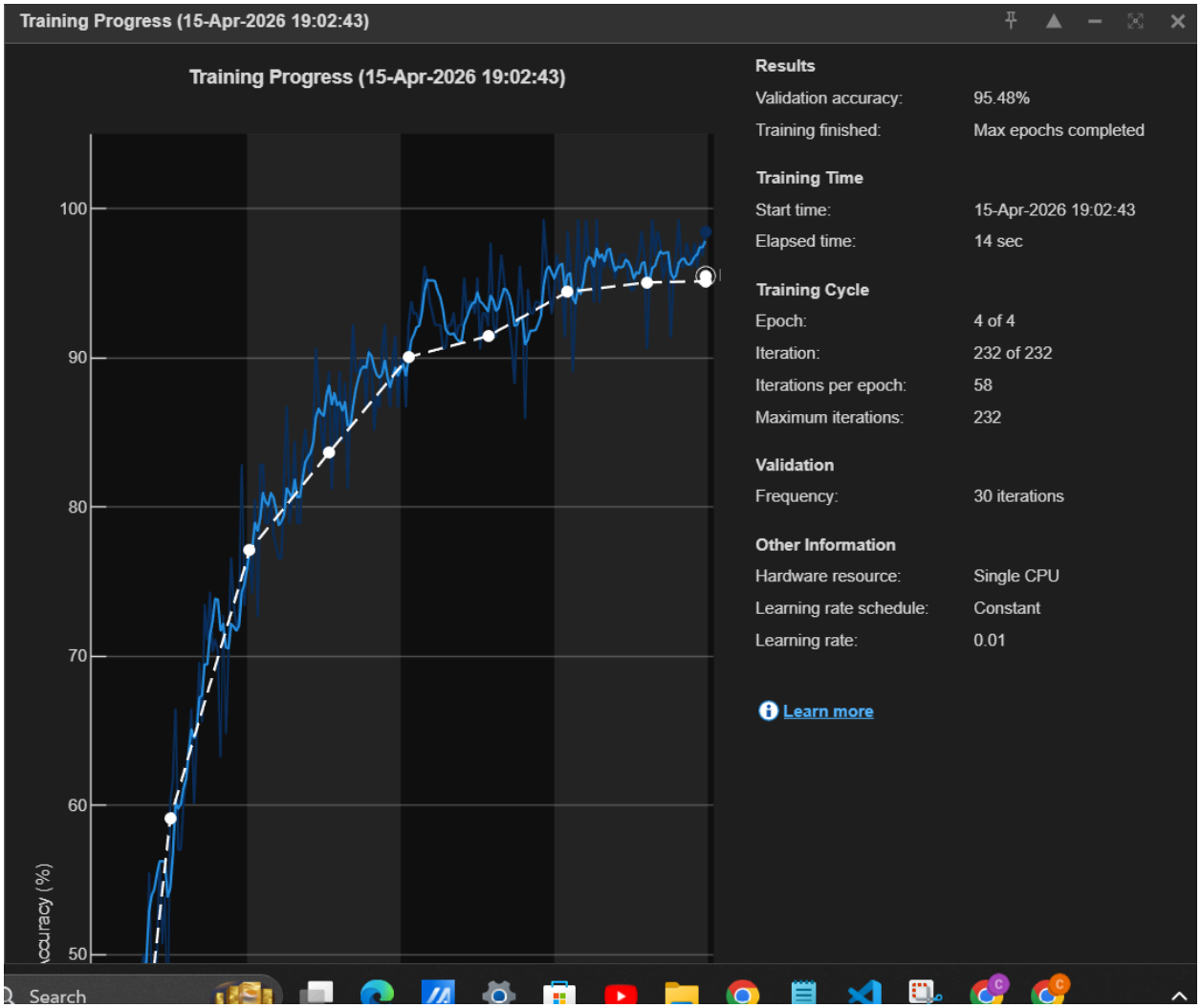
```
>> train_digit_cnn
    Label     Count
    _____     _____

      0       1000
      1       1000
      2       1000
      3       1000
      4       1000
      5       1000
      6       1000
      7       1000
      8       1000
      9       1000

Image Size:
    28     28

Validation Accuracy: 95.48%
```





In conclusion, the CNN model effectively captures spatial hierarchies in image data, achieving high classification accuracy on the Digit Dataset. Its ability to automatically learn relevant features from raw pixels makes it superior to traditional methods for handwritten digit recognition.